

Definition

Reinforcement Learning (RL) is a type of artificial intelligence where a computer program learns to make decisions through trial and error. It does not use ready-made answers; instead, it interacts with its surroundings and learns from the positive or negative feedback it receives.

Key Points

- The Agent: The AI learner that makes choices and takes actions.
- The Environment: The surroundings or world that the agent interacts with.
- The Reward: The positive feedback or points given when the agent does something right.

How It Works

1. The agent looks at its current situation and takes an action.
2. The environment then changes, and the agent receives a reward for a good move or a penalty for a mistake.
3. Over time, the agent changes its strategy to repeat actions that earn the highest total points.

Advantages

1. No data needed: It learns by doing, so it does not require massive datasets of pre-labeled answers.
2. Handles change: It excels at making decisions in unpredictable, dynamic environments.

Short Example

A self-driving car tries to navigate a street. It gets "points" for staying safely in its lane and loses points if it gets too close to a curb, quickly learning how to drive smoothly.

Definition

PAC Learning is a math framework that proves an AI will probably (with high confidence) learn a concept that is approximately correct (with low error).

Key Points

- Probably: High chance the AI successfully learns.
- Approximately: Small amount of error allowed.
- Data Bound: Calculates the exact number of examples needed.

How It Works

1. The AI trains on a random set of data.
2. The PAC framework mathematically calculates the minimum number of samples required to guarantee the AI will perform accurately on new data.

Advantages

1. Proven reliability: Gives mathematical guarantees for AI success.
2. Saves resources: Tells engineers exactly how much data to collect.

Short Example

An AI learns to draw a bounding box around faces. PAC calculations prove exactly how many photos

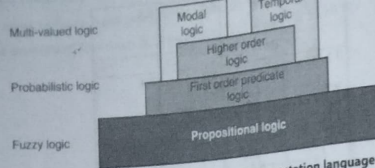


Fig. 4.3.2 : Logic as Knowledge Representation language

- Propositional logic can be considered as fuzzy logic level, where rules are values between range of 0 and 1. Next level is also called as probabilistic logic level using which first order predicate logic is implemented.
- In this Fig. 4.3.2 there are two more levels above higher order logic respectively. All these types of logic are monotonic logic levels and they consist of modal logic and temporal logic respectively. All these types of logic are basic building blocks of intelligent systems and they all use reasoning in order to represent sentences. Hence reasoning plays a very important role in AI.

4.4 Representation of Knowledge using Rules

University Question

Q. Explain various method of knowledge representation techniques.

MU - May 13, Dec. 14, May 15, Dec. 19

- Knowledge can be considered to be represented at generally two levels :

- Knowledge level :** This level describes the facts.
- Symbol level :** This level deals with using the symbols for representing the objects, which can be manipulated in programs.

- Knowledge can be represented using the following rules :

- Logical representations
- Production rule representations.
- Semantic networks
- Frame representations

(a) Logical representation

- The logical representations are mostly concerned with truth of statements regarding the world. These statements are most generally represented using statements like TRUE or FALSE.
- Logic is successfully used to define ways to infer new sentences from the existing ones. There are certain logics that are used for the representation of information, and range in terms of their expressiveness. There are logics that are more expressive and are more useful in translation of sentences from natural languages into the logical ones. There are several logics that are widely used :
 - Propositional logic :** These are restricted kinds that make use of propositions (sentences that are either true or false but not both) which can be either true or false. Proposition logic is also known as propositional calculus, sentential calculus or boolean algebra.

(a) viorics are blue.

Sentence	Truth Value	Proposition
Sky is blue	true	yes
Roses are red	true	yes
$2 + 2 = 5$	false	yes

- First Order Predicate Logic :** These are much more expressive and make use of variables, constants, predicates, functions and quantifiers along with the connectives explained already in previous section.
- Higher Order Predicate Logic :** Higher order predicate logic is distinguished from first order predicate logic by using additional quantifiers and stronger semantics.
- Fuzzy Logic :** These indicate the existence of in between TRUE and FALSE or fuzziness in all logics.
- Other Logic :** These include multiple valued logic, modal logics and temporal logics.

(b) Production Rule Representation

- One of the widest used methods to represent knowledge is to use production rules, it is also known as IF-THEN rules.

Syntax :

IF condition THEN action
 IF premise THEN conclusion
 IF proposition p1 and proposition p2 are true
 THEN proposition p3 is true

Example :

IF pressure is high, THEN volume is small.
 IF the road is slippery, THEN driving is dangerous.

- Some of the benefits of IF-THEN rules are that they are modular, each defining a relatively small part, at least in principle, independent piece of knowledge. New rules may be added and old ones deleted usually independently of other rules.
- Production rules are simple but powerful forms of representing knowledge. They provide flexibility for combining procedural and declarative representations in a unified manner. The major advantage of production rules is that they are modular, independent of other rules with the provision for addition new rules and deleting other ones.

(c) Semantic networks

- These represent knowledge in the form of graphical networks, since graphs are easy to be stored inside programs as they are concisely represented by nodes and edges.
- A semantic network basically comprises of nodes that are named and represent concepts, and labelled nodes representing relations between concepts. Nodes represent both types and tokens.
- For example, the semantic network in Fig. 4.4.1 expresses the knowledge to represent the following info

- Tom caught a fish.
- Tom is grey in color.
- Tom is owned by Sam.
- Tom is a Mammal.
- Fish is an Animal.
- Cats love Milk.
- All mammals are animals.

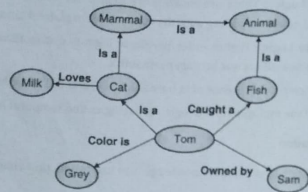


Fig. 4.4.1

- **Conceptual Graph** : It is a recent scheme used for semantic network, introduced by John Sowa, has a finite, connected, bipartite graph. The nodes represent either concepts or conceptual relations. It differs from the previous method that it does not use labelled arcs. **For example** : Ram, Laxman and Bharat are Brothers or cat color is grey can be represented as shown.

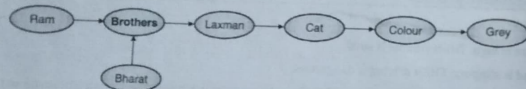


Fig. 4.4.2

(d) Frame Representation

- This concept was introduced by Marvin Minsky in 1975. They are mostly used when the task becomes quite complex and needs more structured representation. More structured the system becomes more would be the requirement of using frames which would prove beneficial. Generally frames are record like structures that consists of a collection of slots or attributes and the corresponding slot values.
- Slots can be of any size and type. The slots have names and values (subfields) called as facets. Facets can have names or numbers too. A simple frame is shown in the Fig. 4.4.2 for a person Ram,

- { Ram }
- { PROFESSION (VALUE professor)
- { AGE (VALUE 50) }

- { WIFE (VALUE sita) }
- { CHILDREN (VALUE luv kush) }
- { ADDRESS (STREET (VALUE 4C gb road)) }
- { CITY (VALUE banaras) }
- { STATE (VALUE mh) }
- { ZIP (VALUE 400615) }

4.4.1 Ontology

Ontology

- Ontology is study about what kind of things or entities exist in the universe. In AI, ontology is the specific set of conceptualizations, used to help programs and humans to share knowledge about a particular domain. In turn ontology is a set of concepts, like entity, relationships among entities, events that are expressed in a unified way in order to create a vocabulary for information exchange.
- An ontology should also enable a person to verify what a symbol means. That is, given a concept, they want to be able to find the symbol, and, given the symbol, they want to be able to determine what it means. Typically, it specifies what types of individuals will be modelled, specifies what properties will be used, and gives some axioms that restrict the use of that vocabulary. Ontologies are usually written independently of a particular application and often involve a community to agree on the meanings of symbols
- For example : Consider a map showing hotels, railway station, buildings, schools, hospitals in a particular locality. In this map the symbols used to indicate these entities are enough to describe them. Hence the community who knows the meaning of these symbols can easily recognize it. Hence that becomes ontology of that map. In ontology, it may define a building as human-constructed artifacts. It may give some restriction on the size of buildings so that shoeboxes cannot be buildings or that cities cannot be buildings. It may also state that a building cannot be at two geographically dispersed locations at the same time.

4.5 Propositional Logic (PL)

- **Propositional Logic (PL)** is simple but powerful for some artificial intelligence problems. You have learnt simple mathematical logic in which uses atomic formulas are used. (Atomic formula is a formula that has no strict no formulas). Atomic logic formulas are called propositions.
- In case of artificial intelligence propositional logic is not categorized as the study of truth values, but it is based on the relativity of truth values. (i.e. The relationship between the truth value of one statement to that of the truth value of other statement)

4.5.1 Syntax

Basic syntax followed by the propositional logic can be given as follows :

- Propositional symbols are denoted with capital letters like : A, B, C, etc.
- Propositional logic constants have a truth value generally truth values have a crisp nature (i.e. 0 (false) or 1 (true)). But for fuzzy logic truth values can vary in the range of 0 and 1.
- Propositional logic make use of wrapping parenthesis while writing atomic sentence. It is denoted as $\neg$.

Note :

1. Typically, $\Rightarrow$ is the main connective with $\forall$

Example : "Everyone at MU is smart" is represented as

$$\forall x \text{ At}(x, \text{MU}) \rightarrow \text{smart}(x)$$

2. Typically, $\wedge$ is the main connective with $\exists$

Example : Someone killed the cat and is guilty.

$$\exists x \text{ killed}(x, \text{cat}) \wedge \text{guilty}(x)$$

o **Equality** : $\text{term}_1 = \text{term}_2$ is true under a given interpretation if and only if term_1 and term_2 refer to the same object

o Example : Richard has at least two brothers

$$\exists x, \exists y \text{ Brother}(x, \text{Richard}) \wedge \text{Brother}(y, \text{Richard}) \wedge \neg (x = y)$$

4.7 Comparison between Propositional Logic and First Order Logic

Sr. No.	Propositional logic (PL)	Predicate logic (FOL)
1.	PL can not represent small worlds like vacuum cleaner world.	FOL can very well represent small worlds' problems.
2.	PL is a weak knowledge representation language	FOL is a strong way of representing language.
3.	Propositional Language uses propositions in which the complete sentence is denoted by a symbol.	FOL uses predicated which involve constants, variables, functions, relations.
4.	PL cannot directly represent properties of individual entities or relations between individual entities. e.g. Meera is short.	FOL can directly represent properties of individual entities or relations between individual entities using individual predicates using functions. E.g. Short(Meera)
5.	PL cannot express specialization, generalizations, or patterns, etc. e.g. All rectangles have 4 sides.	FOL can express specialization, generalizations, or patterns, etc. Using relations. E.g. no_of_sides(rectangle, 4)
6.	PL is a foundation level logic.	FOL is a higher level logic.
7.	PL is not sufficiently expressive to represent complex statements.	FOL can represent complex statements.
8.	PL assumes the world contains facts	FOL assumes the world contains objects, relations, functions like natural language.
9.	In PL Meaning of the facts is context-independent unlike natural language.	In FOL Meaning of the sentences is context dependent like natural language.
10.	PL is declarative in nature.	FOL is derivative in nature.

evaluated to TRUE successor will be true. It is used in sentences.

- Example : The sentence "If the bag is of blue colour, I will buy it." Will be represented as colour (bag, blue) $\rightarrow$ buy(bag).

✓ Quantifiers

- ✓ Apart from these connectives FOPL makes use of quantifiers. As the name suggests they quantify the number of variables taking part in the relation or obeying the rule.

✓ 1. Universal Quantifier ' $\forall$ '

- Pronounced as "for all" and it is applicable to all the variables in the predicate
- " $\forall x A$ " means A is true for every replacement of x.
- Example: "Every Gorilla is Black" can be represented as :

$$\forall x (\text{Gorilla}(x) \rightarrow \text{Black}(x))$$

✓ 2. Existential Quantifier ' $\exists$ '

- Pronounced as "there exists"
- " $\exists x A$ " means A is true for at least one replacement of x.
- Example: "There is a white dog" can be represented as,

$$\exists x (\text{Dog}(X) \wedge \text{white}(X))$$

* What is Planning in AI? :

- Planning in AI is the process of coming up with a series of actions or procedures to accomplish a particular goal.
- Planning in AI can be defined as a problem that needs decision making by Intelligent system to accomplish the given target.
- The Intelligent System can be a Robot or a Computer prog.
- AI planning comes in different types, each suitable for a particular situation. Popular different types of planning in AI include:

- 1) Classical Planning - A series of actions created to accomplish a goal in a predetermined setting. It assumes that everything is static and predictable.
- 2) Hierarchical Planning - By dividing large problems into smaller ones, hierarchical planning makes planning more effective.
- 3) Temporal Planning - Planning ensures that the plan is workable within a certain time limit by taking into account the duration of tasks.

- Challenge in AI Planning:

- 1) Complexity - Due to wide state space, multiple possible actions, planning in complicated domain can be difficult.
- 2) Uncertainty - One of the biggest challenges in AI planning is overcoming uncertainty as actions result might not be anticipated.
- 3) Scalability - Large scale issues must be handled by planning systems.

* Major Approaches for Planning :-

- There are many approaches to solve planning problems.
- Following are the few major approaches used for planning :-

- Planning with ~~Space~~ State Space Search
- Partial Ordered Planning (POP)
- Hierarchical planning / Hierarchical decomposition (HTN planning).
- Conditional planning
- Planning with operation.

* Partial Ordered Planning (POP) :-

- In case of POP, ordering of actions is partial.
- Also Partial Ordered planning does not specify which action will come first out of the two actions which are placed in plan.
- Here sequence of action is not strictly linear.
- This allows flexibility and parallelism in execution.
- Characteristic :

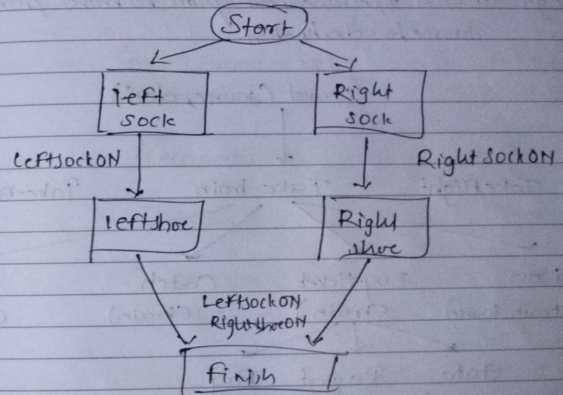
1. Plans are represented as partially ordered set of actions.
2. "Causal links" are used to express the necessity that one action produces a condition required by another.
3. "Open" pre conditions are addressed progressively during planning.
4. POP avoids committing to an action seq. too early.

- Advantage :

1. Avoid unnecessary constraints in plan execution.
2. More efficient in domains where multiple agents are involved.
3. Allows actions to be executed in parallel if they are independent.

- Example :

Take a example of Wearing shoe

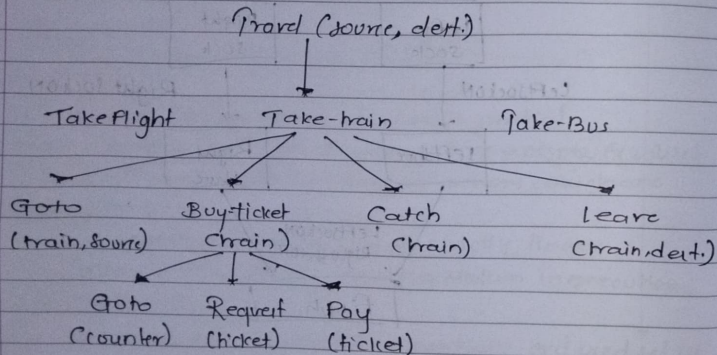


* Hierarchical Planning :-

(HTN - Hierarchical task Network).

- Hierarchical Planning is also called as Plan decomposition.
- Generally, Plans are organized in a hierarchical format.
- Hierarchical planning involves breaking down high-level tasks into smaller, more manageable subtasks.
- It can be denoted with help of links betn various task of different levels of hierarchy. This is called as operator expansion.

- eg, move (x,y,z)
 pickup (x,y) putdown (x,z)
 fig: operator expansion
- Advantage :
 1. More Scalable for large and complex domains
 2. Mirror human problem-solving approaches.
- Eg, Create a Hierarchical plan to travel from one source to destination.



* Difference: —

Feature	Hierarchical Planning	Partial Order Planning (POP)
Core Approach	Top-down, goal decomposition	Bottom-up, Causal-link building
Knowledge Base	Requires domain-specific expert methods	Requires basic ordering rules
Searchspace	Small, bounded by hierarchy.	Large, searches all combinations
	Recipe-driven	flexibility driven
	Strict order	flexible order
Best used for	Complex, predictable workflow flows,	Concurrent, independent operations.

* Difference: —

Partial Ordered Planning (POP)

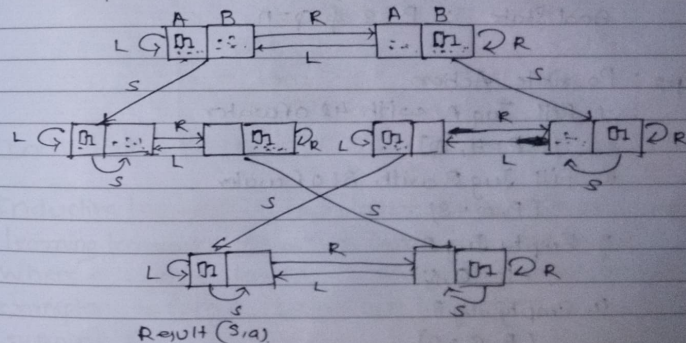
Total Ordered Planning (TOP)

- | | |
|--|--|
| 1. Planning technique where the order of actions is partially defined. | 1. Planning technique where all actions are arranged in a strict total sequence. |
| 2. Highly flexible | 2. Less flexible |
| 3. Plans are represented as partially ordered set of actions. | 3. Plans are represented as totally ordered list of actions. |
| 4. Non-linear planning | 4. Linear planning. |
| 5. Supports parallelism | 5. Does not support parallelism. |
| 6. Easy to insert new steps. | 6. Hard to insert steps. |
| 7. Used in Complex AI planning systems | 7. Used in Simpler Environment. |

* Formulate State Space Search for Vacuum World Cleaner: —

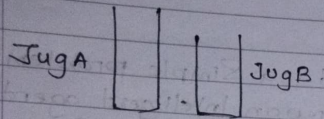
- The Vacuum Cleaner World is a simple problem used in AI to demonstrate how an intelligent agent performs State Space Search to clean an environment.
- The environment consists of rooms that may be clean or dirty and the vacuum agent must clean all rooms efficiently.

1. Initial State: Any state can be designated as initial state.
2. Actions: In this environment, each state has just three actions: Left, Right, and Suck.
3. Successor function:



4. Goal test: This checks whether all the squares are clean.
5. Path cost: Each step costs 1 so the path cost is the number of steps in the path.

* Water-jug Problem :



Goal : Fill Jug A with 2L water.

Neither has any measuring markers.

Step 1 : State Description

Let $P = \text{Jug A}$ &

Let $Q = \text{Jug B}$

$0 \leq P \leq 4$

$0 \leq Q \leq 3$

Initial State : $P = 0 \text{ \& } Q = 0$

Goal State : $P = 2 \text{ \& } Q = n$

Step 2 : Possible Action

1. Fill Jug A with 4L of water

$[P = 4, Q]$

2. Fill Jug B with 3L of water

$[P, Q = 3]$

3. Empty Jug A

$[P = 0, Q]$

4. Empty Jug B

$[P, Q = 0]$

5. Transfer complete water from B to A.

$[P, Q] \rightarrow [P + Q, 0]$

6. Transfer complete water from A to B.

$[P, Q] \rightarrow [0, P + Q]$

7. Transfer some water from B to A till A gets full.

$[P, Q - (4 - P)]$

13/5/26

8. Transfer some water from A to B till B gets full
 $[P - (3 - Q), Q]$

Steps : Solution

Initial State : $P = 0 \text{ \& } Q = 0$

↓ (Rule 2)

$P = 0, Q = 3$

↓ (Rule 5)

$P = 3, Q = 0$

↓ (Rule 2)

$P = 3, Q = 3$

↓ (Rule 7)

$P = 4, Q = 2$

↓ (Rule 3)

$P = 0, Q = 2$

↓ (Rule 5)

$P = 2, Q = 0$

↓

Goal State

* Inductive learning :

- Inductive learning is foundational approach or supervised learning technique because it learns through examples.
- Where a system learns by generalizing from specific examples to form a broader rule or pattern.

- Key points :

- Learns from training eg.
- Generalizes from observed data.
- Used widely in Machine Learning.

- How it works :

- Observe - AI looks at 1000 of labelled data eg.
- Detect - It identifies hidden patterns and correlates
- Generalize - Create formula/model based on those ptns.

- Eg: learning SPAM vs non-spam email from past labeled data email.

- Adv:
- can handle new & unseen situations.
 - flexible & adaptive.

* Rule Learning:

- Rule learning is a learning method in which an AI system memorizes facts or solutions exactly as they are without generalization.

- Key point:

- Store information directly.
- No reasoning or pattern discovery.
- Works like a lookup table.

- How it works:

- I/p is matched with stored data.
- corresponding o/p is retrieved.

- Eg: Remembering Multiplication table

- Adv:

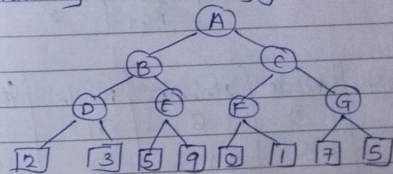
- fast retrieval
- Simple Implement.

Reinforce PAC

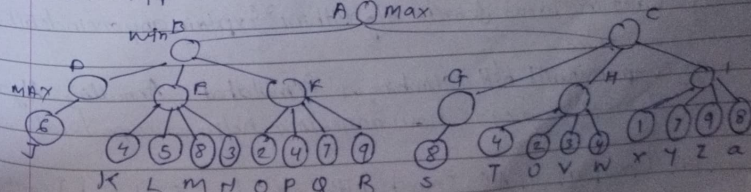
Module 3

Problem Solving

- ✓ 1] What do you understand by Informed and Uninformed Search Method? Explain in detail. *MVA, May '23*
- ✓ 2] What do you understand by Min-Max Search & Alpha beta Search? Explain in detail. *MVA, Dec '23*
- ✓ 3] Explain Hill Climbing Algorithm & problems that occur in it with its limitation. *MVA, May '25, May '23, May '24*
- ✓ 4] Explain A* search algorithm with suitable example. Justify if this is informed or uninformed search? *MVA, May '25, Dec '23*
- ✓ 5] Explain Depth limit Search and Depth first iterative Deepening Search. *MVA, Dec '24*
- 6] What is game playing algorithm? Draw a game tree for Tic-Tac-Toe problem. *MVA, May '24*
- ✓ 7] Differentiate: Informed Search & Uninformed Search Algorithm. *MVA, May '24*
- 8] Shortnote: Simulated Annealing *MVA, May '23*
✓ Genetic Algorithm *MVA, May '25, May '23, Dec '23*
- 9] What do you understand by α - β pruning? Apply alpha beta pruning on following graph. *May '25, May '24, Dec '24*



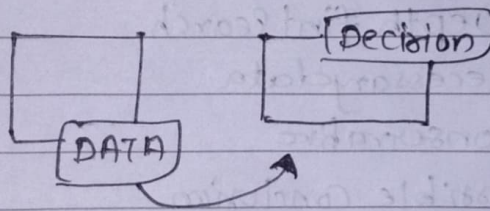
- 10] Perform α - β pruning on full graph, clearly indicating the α & β cut and final value of root node. *May '25*



Forward Backward

Q7 Forward Chaining : —

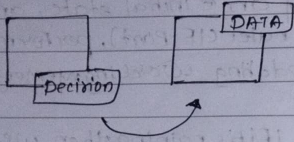
- When based on available data a decision is taken, then the process is called forward Chaining.
- a.k.a as Data driven Interference technique.
- Works from an initial state, and by looking at the premises of rules (IF-part), perform the actions (THEN part), possibly updating working memory.
- Foreg., "If it is raining then, we will take umbrella".
Here, "It is raining" is data and "we will take an umbrella" is a decision.
- This means it was already known that it's raining that's why it decided way to take umbrella.
- This process is forward chaining.



Q7 Backward Chaining : —

- If based on decision the initial data is fetched, then it is called as Backward Chaining.
- a.k.a as Goal-driven Interference
- Works towards a final state, and by looking at working memory to see if goal already there, if not look at actions (THEN-part) of rules that will establish goal, and setup subgoals for achieving premises of rules (IF-part).

- For eg., "If while going out one has taken umbrella. ^{raining} Here, "taking umbrella" is a decision based on which data is generated that "it's raining".
- This process is Backward chaining.



* Difference :-

Attribute	Backward chaining	forward chaining
a.k.a	Goal driven	Data-driven
Reasoning	Top-down reasoning	Bottom-up reasoning
Type of Search	Depth first search	Breadth first search
Aims for	Necessary data	Any conclusion
Approach	Conservative	Opportunistic
Starts from	Possible conclusion	New data

* Uninformed Search :-

- a.k.a. Searching without Information or Blind Search.
- Has info. abt only two things: Goal State and start state.
- Doesn't have any guide to take it to goal state.
- Can distinguish betn Goal State & Non-goal state.
- More time consumption and High chances of optimal solution.
- Eg., BFS, DFS, etc., UCS, DLS, IDS.

- Uninformed Search Methods are useful when we don't have any additional heuristic information but they are usually inefficient for large or complex problems due to their blind exploration.

* Informed Search Method :-

- a.k.a. Heuristic Search Methods.
- The main idea is to generate additional information about the search state space using the knowledge of problem domain, so that the search becomes more intelligent and efficient.
- This knowledge is often expressed as Heuristic function $h(n)$, which estimates the cost or distance from the current node to goal node.
- Eg., Greedy Best first search, A* Search Algorithm.

* Difference :-

Parameter	Informed search	Uninformed search
Utilizing knowledge	It uses knowledge during process of searching.	Does not req. using any knowledge during searching.
Speed	find sol ⁿ quicker	find sol ⁿ much slower
Consumption of time	Consumes much less time.	Consumes comparatively more time.
Cost	Much lower	Comparatively higher
Efficient	More efficient	Less efficient.
Length	Shorter	longer
Eg.	Graph Search, Greedy search	DFS, BFS

* Wumpus World Environment:

- similar to vacuum world problem, we have Wumpus World problem.
- Early computer had a game called "Hunt the Wumpus" based on that Wumpus was developed.
- It was originally written in BASIC.
- Wumpus is a Map-based game.
- Wumpus World is a grid-based environment (usually 4x4) where an intelligent agent (Player) explores the world to find gold, escape, while avoiding deadly hazards like pit and Wumpus monster.
- Things are done either horizontally/vertically not crossly.

1. Introduction

Wumpus, Stench, Agent, Pit, Breeze, Gold, Arrow.

4.1 STENCH	4.2	4.3 Breeze	4.4 PIT
3.1 	3.2 Breeze Stench 	3.3 PIT	3.4 Breeze
2.1 STENCH	2.2	2.3 Breeze	2.4
1.1 	1.2 Breeze	1.3 PIT	1.4 Breeze

2. Awards and Punishment

- +100 points if Agent comes out with gold.
- 1 point on agents every action.
- 10 point if arrow is used.
- 200 point if agent dies.

• PEARs properties of Wumpus World

1. Performance Measured

- +100 pts if Agent comes out with gold.
- -1 pts on agent every action.
- -10 point if arrow is used.
- -200 pts if Agent dies.

Environment's prop's

2. Environment

- Empty rooms
- Rooms with Wumpus
- Stench rooms
- Breezy rooms
- Room with gold
- Arrow

1. fully observable

2. Deterministic

3. Episodic

4. Static

5. Discrete

6. Single Agent

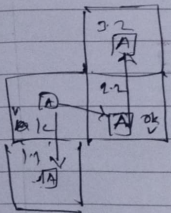
3. Sensor

- Camera
- Odour sensor
- Audio sensor

4. Actuator

- Motor (to move L.R)
- Robotic arm to grab gold
- Robotic mechanism to shoot arrow

- Step 1: Initially the player in room.
- Step 2: Move to Room 1.2. Breeze is there, it can be predicted that there is pit (horiz, vertically). Don't safe.
- Step 3: Player in room 2.1 where it receives stench, which means there is Wumpus in neighbouring room either (2.2) or (3.1). As we know no breeze in 2.2; don't pit there and also no stench so no wumpus.
- Step 4: Safe to move in (2.2) when it comes to room 2.2 it understands that rooms 2.3 and 3.2 are also safe to move in.
- Step 5: move to room (3.2) grab the gold and go back. without being killed.
- Step 6: Thus we won WOMPUS WORLD.



* Genetic Algorithm

- GA are adaptive heuristic Search Algorithm based on Evolutionary idea of natural selection and genetics.
- Although randomized, GAs are by no means random, instead they exploit historical information to direct the search for better performance within search space.
- Genetic Programming (GP) is a type of evolutionary algo. and an extension of Genetic Algo (GA), inspired by process of natural selection in biological evolution.
- While Genetic Algos (GA) evolve soln encoded as strings (often binary); Genetic programming evolves actual program or functional expressions to solve a given problem.
- Terminologies of GA:
 - Gene - represent smallest unit of info in PD
 - Chromosome - represents the combinations possible soln
 - Encoding - depends on type of problem.
 - Population - pool of individuals (chromosome) that will be sampled for selection and evaluation.
 - Reproduction - process of creating new individuals called offspring from parent population.

- Genetic Operation:

- **Selection** - process of choosing fittest individual to form a new gen.
- **Crossover** - Two parent prog. combine parts of their structure to create offspring.
- **Mutation** - A small change in individual structure to introduce variability and avoid local minima.

Algorithm

1. [start]
2. [fitness]
3. [New pop.]
4. [Selection]
5. [Crossover]
6. [Mutation]
7. [Accepting]
8. [Replace]
9. [Test]
10. [Loop]

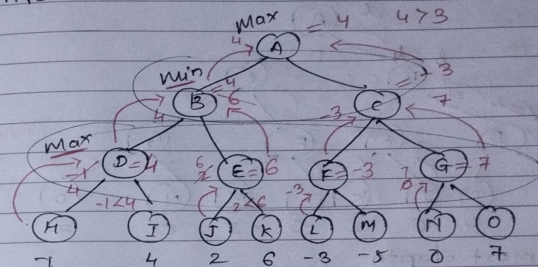
- Eq. 11

Parent Chromosome	Chromosome A	1 5 3 2 6 4 7 9 8
	Chromosome B	8 5 6 7 2 3 1 4 9

Child chromosome / chromosome 1 / 909987837

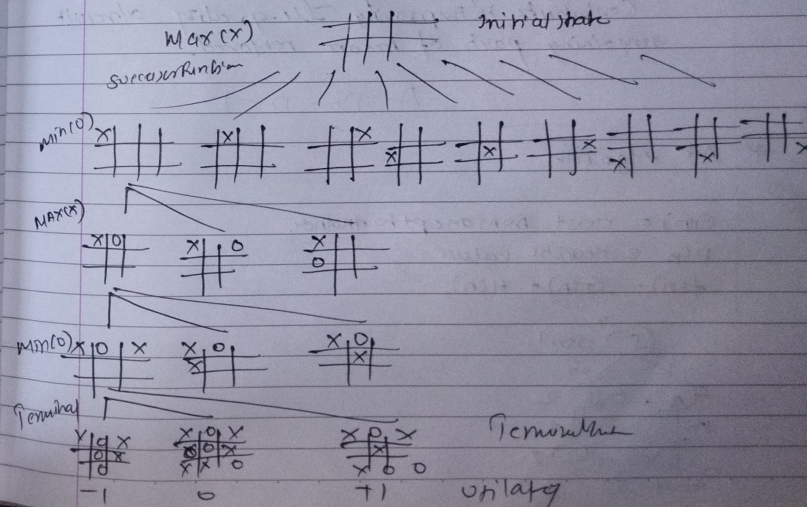
* Minimax Algorithm :

- It evaluates decision based on present status of game.
- This algo. needs deterministic environment with exact info



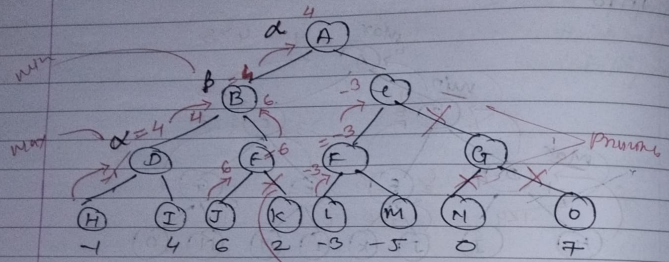
- most left hand side depth. first go

so path $\Rightarrow A \rightarrow B \rightarrow D \rightarrow I$



Alpha beta pruning:

- Pruning = cutting off, clipping a branch in search tree

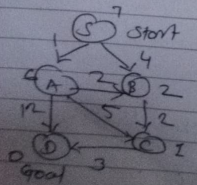


- Leftmost depth
- Timespace gain
- Not exploring all nodes
- Pruning is only those parts of tree which contribute in improving the quality of result remaining parts of tree are removed.

A-B-D-1

* A* Algorithm.

$G(n)$ = cost from concept to node n.
 $H(n)$ = Heuristic value
 $f(n) = G(n) + H(n)$



$$f(n) = g(n) + h(n)$$

$$= 0 + 7$$

$$f = 7$$

$$S \rightarrow A$$

$$f(n) = g(n) + h(n)$$

$$= 1 + 6$$

$$= 7$$

$$S \rightarrow B$$

$$f(n) = g(n) + h(n)$$

$$= 4 + 2$$

$$= 6$$

$$S \rightarrow B \rightarrow C$$

$$f(n) = g(n) + h(n)$$

$$= 6 + 1$$

$$= 7$$

$$S \rightarrow A \rightarrow B$$

$$f(n) = g(n) + h(n)$$

$$= 3 + 2$$

$$= 5$$

$$S \rightarrow A \rightarrow C$$

$$f(n) = g(n) + h(n)$$

$$= 6 + 1$$

$$= 7$$

$$S \rightarrow A \rightarrow D$$

$$f(n) = g(n) + h(n)$$

$$= 13 + 0$$

$$= 13$$

$$S \rightarrow A \rightarrow B \rightarrow C$$

$$f(n) = g(n) + h(n)$$

$$= 5 + 1$$

$$= 6$$

$$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$$

$$f(n) = g(n) + h(n)$$

$$= 8 + 0$$

$$= 8$$

$$S \rightarrow B \rightarrow C \rightarrow D$$

$$f(n) = g(n) + h(n)$$

$$= 9 + 0$$

$$= 9$$

$$S \rightarrow A \rightarrow C \rightarrow D$$

$$f(n) = g(n) + h(n)$$

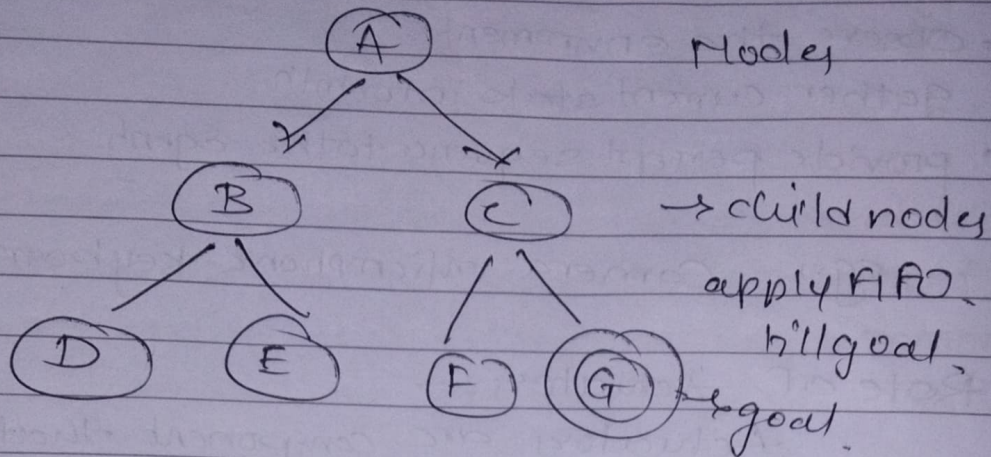
$$= 9 + 0$$

$$= 9$$

path which min,
 $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \Rightarrow 8$

- 2X $S \rightarrow A \Rightarrow 7$
- 1X $S \rightarrow B \Rightarrow 6$
- 5X $S \rightarrow B \rightarrow C \Rightarrow 7$
- 3X $S \rightarrow A \rightarrow B \Rightarrow 5$
- 6X $S \rightarrow A \rightarrow C \Rightarrow 7$
- * $S \rightarrow A \rightarrow D \Rightarrow 13$
- 4X $S \rightarrow A \rightarrow B \rightarrow C \Rightarrow 6$
- * $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \Rightarrow 8$
- * $S \rightarrow B \rightarrow C \rightarrow D \Rightarrow 9$
- * $S \rightarrow A \rightarrow C \rightarrow D \Rightarrow 9$

Q. BFs part 1



{A} → AX

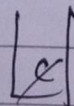
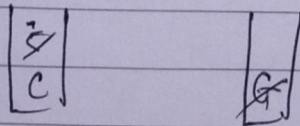
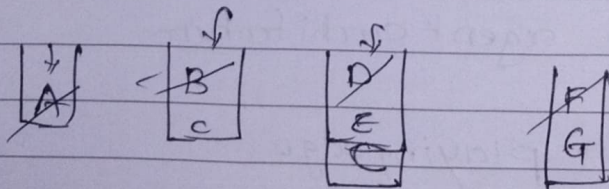
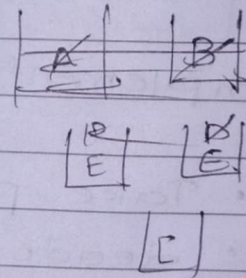
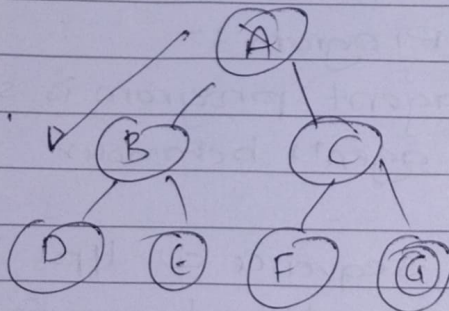
{B, C} → BX

{D, E, F} → CX

{D, E, F, G} → BEF XXX

{G} → goal

Q. DFS



15/4/26

* Answers for UT 2 QB:—

10Q] Differentiate: Hill Climbing vs A* Algorithm.

Parameters	Hill Climbing	A* Algorithm
Type of Search	Local Search Algo.	Informed Search Algo
Approach	Greedy (chooses best + immediate neighbor)	Uses both actual cost + heuristic.
Evaluation (F^n)	Uses only heuristic $F^n = h(n)$	Uses $F(n) = g(n) + h(n)$
Search Nature	Does not explore entire Search space.	Explores multiple paths systematically.
Optimality	Not guarantee to find Optimal soln	Guarantee Optimal
Completeness	Not complete	Complete.
Backtracking	No backtracking.	Allow
Memory usage	Little Memory req'd	Large Memory req'd
Speed	Faster	Slower than hill climbing.
Slow Quality	May give Sub-optimal soln	Gives optimal soln
Use	Optimization problems	Pathfinding Shortest Path Prblm.

1Q] Explain "Depth Limited Search Algorithm" with suitable examples.

- Depth Limited Search Algorithm is Search Algorithm where we explore the Search tree only upto fixed depth limit (L).
- It prevents DFS from going infinitely deep (which is a major problem in infinite trees)

Need DLS

In normal DFS,

- It may go into infinite depth.
- It may never find soln, even if it exists.

DFS solve this by:

- Adding a maximum depth limit.
- Stopping Search beyond that level.

Basic working

- Starting from root node
- Explore nodes depth-wise (like DFS).
- Stop when

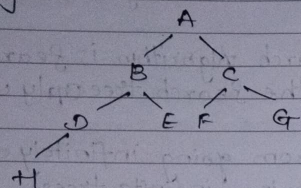
Goal is found ✓

Depth limit is reach x

• Algorithm:—

1. Start at root node.
2. If node is goal → return Success.
3. If depth == limit → return cutoff.
4. Else:
 - Expand node.
 - Apply DLS to each child.
5. If any child return success → return success.
6. Else → return failure.

foreg.,



Case 1 : Depth Limit = 3 levels

- Level 0 $\rightarrow$ A.
- Level 1 $\rightarrow$ B, C
- Level 2 $\rightarrow$ D, E, F, G.

$\therefore$ Node H is at lvl 3, so it will not be explored.

Traversal: (DFS style)

A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E $\rightarrow$ C $\rightarrow$ F $\rightarrow$ G.

Case 2 : Depth limit = 3

Now H is within limit

Traversal

A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ H $\rightarrow$ E $\rightarrow$ C $\rightarrow$ F $\rightarrow$ G

$\therefore$ Goal can be found if it is H.

Adv.

1. Prevents Infinite loops
2. Saves Memory (like DFS)
3. Useful when soln depth is known.

Disadv.

1. If limit is too small $\rightarrow$ may miss soln.
2. If limit too large $\rightarrow$ behave like DFS.
3. Not Optimal (may not give shortest path)

2[8]

Explain Uniform Cost Search Algorithm with suitable examples.

- $\rightarrow$ Uniform Cost Search (UCS) is an Uninformed Search algorithm used in Artificial Intelligence to find the minimum cost path from a Start node to a Goal node.
- UCS always select least expensive path. It guarantees finding the optimal (least cost) soln provided all step costs are non-negative.
- Basic idea
- Instead of expanding nodes level by level (like BFS), UCS expands nodes based on path cost.
- It uses a priority queue where, priority = lowest path cost
- The node with smallest cost is expanded first

• Working principle :

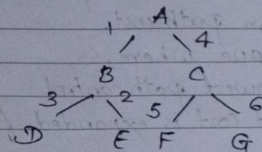
1. Start with Initial node (cost = 0).
2. Insert it into a priority queue.
3. Repeat the following steps :
 - Remove the node with the lowest cost.
 - If it is the goal node $\rightarrow$ return soln.
 - Otherwise, expand the node.
 - Add all child nodes to the queue with updated costs.
4. Continue until the goal node is reached.

Algorithm

Step-by-step procedure

1. Create a priority queue (PQ).
2. Insert start node, with cost = 0
3. While PQ is not empty
 - Remove node with minimum cost.
 - If node is goal $\rightarrow$ return path.
 - Else expand node.
 - Add children with updated path cost.

Eg.



Step-by-step Execution

Step 1: Start at node A.

Cost = 0

Queue: (A, 0)

Step 2: Expand A

Add

B $\rightarrow$ cost = 1

C $\rightarrow$ cost = 4

Queue: (B, 1), (C, 4)

Step 3: Expand B (lower cost)

Add

D $\rightarrow$ 1 + 3 = 4

E $\rightarrow$ 1 + 2 = 3

Queue: (E, 3), (C, 4), (D, 4)

Step 4: Expand E (No children)

Queue: (C, 4), (D, 4)

Step 5: Expand C

Add

F $\rightarrow$ 4 + 5 = 9

G $\rightarrow$ 4 + 6 = 10

Queue: (D, 4), (F, 9), (G, 10)

Step 6: Expand D

Goal reached

Find result

Optimal path A $\rightarrow$ B $\rightarrow$ D

Total cost: 4

UCS always select the minimum cost path, not the shortest in no. of steps.

Adv.

1. Optimal soln

2. Suitable for weighted graph.

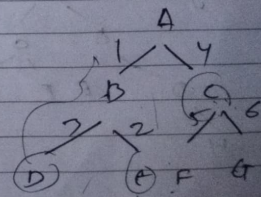
3. Widely used in real-world appln.

Disadv.

1. req. large memory

2. Slow when path cost large

3. Explores unnecessary paths before reaching goal.



3 Q] Explain Deepening first Search Algorithm with suitable example.

→ Iterative Deepening first Search Algo. is search technique in which Depth limited search (DLS) is applied repeatedly with increasing depth limit until the goal node is found.

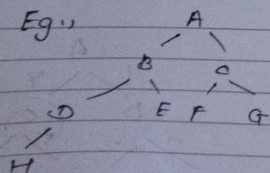
Working Principle

1. Set depth limit $L=0$
2. Apply Depth limited search (DLS)
3. If goal found $\rightarrow$ return soln
4. Else Increase $L=L+1$
5. Repeat until goal is found.

Algo Steps

1. Initialize depth = 0
2. Repeat: perform DLS with given details.
 - a. If result is success $\rightarrow$ return soln
 - Else increase depth.
3. Continue until goal is reached.

Eg.,



Goal node = H

Iteration 1: Depth limit = 0

Visited:

A (Goal not found)

Iteration 2: Depth limit = 1

Visited:

A $\rightarrow$ B $\rightarrow$ C (Goal not found)

Iteration 3: Depth limit = 2

Visited:

A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E $\rightarrow$ C $\rightarrow$ F $\rightarrow$ G (Goal not found)

Iteration 4: Depth limit = 3

Visited:

A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ H (Goal found)

Final result

path A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ H

Goal found at depth = 3

Adv

- 1) Combine benefits BFS & DFS
- 2) Use less memory than BFS.
- 3) Guarantee finding shallowest soln.

Disadv

- 1) Repeat nodes multiple times.
- 2) Slightly more time.

5 Q] What are the Characteristics of Heuristic Search?
Explain properties of Heuristic Search Algorithms.

Heuristic Search uses a heuristic function $h(n)$ that estimate the cost from a node to the goal.

* Characteristic of Heuristic Search: —

1) Use Domain Knowledge:

- It relies on heuristic function $h(n)$ that estimate how close a node is to the goal.
- This makes the search more intelligent and directed.

2) Goal Oriented:

- Instead of exploring blindly it prioritizes nodes that seem closer to the goal.
- Reduces unnecessary exploration.

3) Reduce Search Space:

- By making informed choices it avoids expanding irrelevant nodes.
- Much more efficient than uniformed search in large problem.

4) Not always Optimal, Optimal:

- If the heuristic is not well-designed the Algorithm miss the Optimal soln.

5) Performance Depends on Heuristic Quality:

- Good heuristic $\rightarrow$ fast & optimal
- Poor heuristic $\rightarrow$ behave like uninformed search.

6) Can be greedy or Balance:

7) Informed Search Technique:

- Unlike Blind Search, it has additional info.
- Makes better decision while exploring nodes.

* Properties of Heuristic Search Algorithms: —

① Admissibility:

An Heuristic is admissible if it never overestimates the true cost $h(n) \leq$ actual cost.
Guarantees optimal soln.

② Consistency:

Stronger condition than admissibility.

An heuristic is consistent if:

$$h(n) \leq \text{cost}(n, n') + h(n')$$

③ Completeness:

Algo. find soln if it exist.

Eg., A* is complete (under certain condⁿ)

④ Optimality:

Find the best (least cost) soln.

Achieved when heuristic is admissible.

⑤ Time Complexity:

Depends on quality of heuristic.

Better heuristic $\rightarrow$ fewer nodes explored.

⑥ Space Complexity:

Heuristic algo. stores nodes in memory.

8 Q Explain Hill Climbing and its drawbacks in details.

→ Hill Climbing is an Iterative Search Algo. that starts with an initial soln and repeatedly moves to a neighbouring state with a better value until no further improvement is possible.

- Key idea

- Start from an Initial State.
- Evaluate neighbouring State.
- Move to a best neighbour exists.
- It does not backtrack.

- Drawbacks :-

1. Local Maximum Problem :
Algo may stop at local peak, not global peak.
2. Plateau Problem :
Algo. gets stuck in a flat region where all neighbours have same value.
3. Ridge Problem :
Path to peak is narrow & zig-zag
Algo. cannot follow proper path.
4. No Backtracking :
Once it moves forward, it almost goes back.
5. No Complete :
May fail to find soln
6. No Optimal :
Does not guarantee Best soln.

15 Q Explain forward Chaining Algo. with the help of example.

→

- Forward Chaining Algo. is a data driven inference algo. used in AI.
- It starts with known facts & applies rules to derive new facts until the goal is reached.
- It mainly used
 - Expert System
 - Rule based systems

• Algorithm :-

1. Initialize
Add known facts to working memory.
2. Match Rules
Find rules whose condⁿ (if part)
Match known facts.
3. Fire Rule
Apply the rule & add conclusions (then part)
4. Update
Add new facts to knowledge base
5. Repeat
Continue until goal is reached or rule applies

Eg.,

Given knowledge base
1